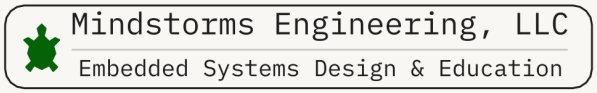
You Don't Need an RTOS (Part 1)

by Nathan Jones



At least, probably not as often as you think you do. Using a preemptive RTOS ("real-time operating system") can help make a system schedulable (i.e. help all of its tasks meet their deadlines) but it comes at a cost: "concurrently" running tasks can interact in unforeseen ways that cause system failures, that dreaded class of errors known as "race conditions". In part, this is because humans have difficulty thinking about things that happen concurrently. Using the venerable superloop can help avoid this problem entirely and, in some cases, it can actually make a system work that wouldn't work at all using an RTOS! Additionally, a "superloop" style scheduler has the benefit of being simple enough for you to write yourself in a few dozen lines of code.

If you're unfamiliar with terms like "deadlines", "schedulability", or "worst-case response time", then I would argue you are almost *definitely* using an RTOS prematurely! If you *are* familiar with those terms, allow me to show you a few ways to improve the superloop that bring its performance nearly on par with an RTOS, with no concern about race conditions.

This is the first in a series of posts lauding the advantages of the simple "superloop" scheduler. Subsequent articles will be linked below as they are published

- Part 1: Comparing Superloops and RTOSes (this article)
- Part 2: The "Superduperloop"
- Part 3: Other Cool Superloops and DIY IPC
- Part 4: More DIY IPC

Contents

- Part 1: Comparing Superloops and RTOSes
- The Problem with Concurrency
    - References
- "In the red corner...!"
- Definition Time
- Critical Assumptions
- Schedulability of a Superloop
- Measuring WCET
    - Using a logic analyzer or oscilloscope
    - Using a serial converter or debug adapter

- Schedulability of an RTOS
    - Assigning Task Priorities
    - The Utilization Test and Harmonic Rate Monotonic Scheduling
- Summary
- References

Part 1: Comparing Superloops and RTOSes

In this first article, we'll compare our two contenders, the superloop and the RTOS. We'll define a few terms that help us describe exactly what functions a scheduler does and why an RTOS can help make certain systems work that wouldn't with a superloop. By the end of this article, you'll be able to:

- Measure or calculate the deadlines, periods, and worst-case execution times for each task in your system,
- Determine, using either a response-time analysis or a utilization test, if that set of tasks is schedulable using either a superloop or an RTOS, and
- Assign RTOS task priorities optimally.

The Problem with Concurrency

Allow me to begin by establishing why we should care: the majority of humans (anywhere from 98-99%) **can't** multitask. (To borrow a line from one of my favorite books, "Statistically, that number includes you. That's how statistics works.") We don't think concurrently, we think *sequentially*. Maybe that's what makes us such great coders and so poor at thinking about all of the ways that our code can possibly interact with itself. Using a preemptive RTOS (and absent any special precautions), a task could be preempted in between *any two* of its machine instructions by *any* higher-priority task. The interactions are too numerous to count and in some small, but non-zero, number of cases this has the potential of causing system errors. These errors, by the fact that they only occur if a certain task gets preempted by another task in *juuuust* the right spot, are called "race conditions" and they're notoriously difficult to diagnose. Better to avoid them entirely, if we can.

Here are a few examples to help prove my point. The code below (modeling a bank transaction) exhibits a race condition; can you find it?

```
transfer1 (amount, account_from, account_to)
{
    if (account_from.balance < amount) return NOPE;
    account_to.balance += amount;
    account_from.balance -= amount;
    return YEP;
}
```

Answer: The lines of code that add or subtract `amount` will be implemented as "read-modify-write" operations on all load-store processors (which is most of them). Imagine the case where `account_to.balance` is 500 and `amount` is 100. The task running this code would first load 500 into a register and add 100 to it, yielding 600. Further, imagine that a higher-priority task is allowed to run *at that exact moment* which adds 200 to that same account; the value in memory would be 700 once the higher-priority task finished. Once the lower-priority task is returned to, it then *overwrites* the value of 700 with 600 (the value in its register)!

The code below (a snippet from a module that implements a Singleton object) also has a race condition; can you find it?

```
if (!initialized)
{
    object = create();
    initialized = true;
}
... object ...
```

Answer: If a task which is attempting to create the Singleton object is preempted *after* the object was created but *before* the flag variable is set to `true` by a higher-priority task which, *itself*, tries to create the Singleton object, then your system now has two Singletons!

Say we try to fix this code using a mutex, like below. No luck! This code *still* has a race condition in it! But where?

```
if (!initialized)
{
    lock();
    if (!initialized)
    {
        object = create();
        initialized = true;
    }
    unlock();
}
... object ...
```

Answer: The problem comes from optimizing compilers which, seeing that there doesn't seem to be a relationship between the lines of code that create the Singleton object and that set the flag variable to `true`, may, in the interest of speed or code size or whatever, hoist the setting of the flag variable *above* the creating of the Singleton object. In that case, it's possible that a task is executing this code and is preempted *after* the flag variable is set to `true` but *before* the Singleton object is created. The currently-running higher-priority task would then be blocked from creating the Singleton object and would, instead, go on to use an object that hadn't yet been created!

It's particularly interesting to me that there would be **no race conditions** in any of the examples above if there was simply no preemption in those systems; using a superloop scheduler as opposed to an RTOS would have literally eliminated the race conditions. This means, to me, that **a preemptive RTOS should be used *only* when it's deemed to be absolutely necessary**, when a superloop scheduler is demonstrably insufficient for the application in question. And if it is, special precautions need to be taken to help prevent possible race conditions (but that's a discussion for another article!).

References

- Fun with Concurrency Problems by Paul Anderson
- The Anatomy of a Race Condition by Matthew Eshleman
- Race Condition vs. Data Race by John Regehr
- What is a race condition? by baeldung
- Race conditions (Embedded Artistry Field Atlas entry)

"In the red corner...!"

Okay, let's have a look at our two contenders! The table below shows how you might program an embedded system using both a superloop and an RTOS. The hypothetical system in question is using a PD (proportional-derivative) closed-loop controller to control "something" and is also implementing some kind of serial interface (to allow the developers to read and modify data as this system is running).

Superloop	RTOS

`volatile unsigned int g_systick = 0;`
`void timerISR(void) // Runs every 1 ms`
`{`
`  g_systick++;`
`}`
`void main(void)`
`{`
`  // Local variables for readSerial task`
`  //`
`  char buf[64] = {0};`
`  int idx = 0;`
`  // Local variables for PD task`
`  //`
`  int kP = 3, kD = 2;`
`  int pastVal = 0;`
	`void main(void)`
	`{`
	`  taskCreate(readSerial, 0);  // Priority 0 (highest)`
	`  taskCreate(PD, 1);  // Priority 1`
	`  startScheduler();  // No return`
	`}`
	`void readSerial(void)`
	`{`
	`  char buf[64] = {0};`
	`  int idx = 0;`
	`  while(1)`
	`  {`
	`  char c = getc();  // Blocking call`
	`  buf[idx++] = c;`

```
    unsigned int PD_period_ms =
500;
    unsigned int PD_next_run = 0;
    // Superloop
    //
    while(1)
    {
        // readSerial task
        //
        if( charAvailable() )  //
Non-blocking call
        {
            char c = getc();   //
Blocking call
            buf[idx++] = c;
            if( c == '\n' )
            {

processCommand(buf);
                memset(buf, 0,
64);
                idx = 0;
            }
        }
        // PD task
        //
        if( PD_next_run -
g_systick >= PD_period_ms )
        {
            int val =
readSensor();
            int output = (kP *
val) + ( kD * (val - pastVal) *
PD_period_ms / 1000 );
            setOutput(output);
            pastVal = val;
            PD_next_run +=
PD_period_ms;
        }
    }
}
```

```
        if( c == '\n' )
        {
            processCommand(buf);
            memset(buf, 0, 64);
            idx = 0;
        }
    }
}
void PD(void)
{
    int kP = 3, kD = 2;
    int pastVal = 0;
    int timeBase_ms = 500;
    while(1)
    {
        int val = readSensor();
        int output = (kP * val) + ( kD
* (val - pastVal) * timeBase_ms /
1000 );
        setOutput(output);
        pastVal = val;
        RTOS_delay(timeBase);
    }
}
```

Pretty similar; the superloop just runs through each task in, well, a "super loop", whereas the RTOS requires that each task be written as separate functions, each with their own infinite loops. The major differences arise because the RTOS code contained two *blocking* function calls: `getc()`

and `RTOS_delay()`. These sorts of function calls are fine in an RTOS, since the scheduler will simply run another task while the first is waiting, but it spells death for the superloop. While the superloop is waiting to receive a character, the `PD` task would have completely stalled; alternatively, while the `PD` task is delaying 0.5 sec the `readSerial` task wouldn't have been capable of processing any incoming characters.

To solve this, we needed to convert those into *non-blocking* function calls. For the `readSerial` task this required finding or writing a function that let us know if a character were available to be read (which, hopefully, is not hard to do). In the case of the `PD` task, that necessitated creating a system clock to keep track of time. (You may also need to make sure that *all* of your `while` loops have some sort of conditional that will allow them to terminate eventually; "`while( (c=getc()) != '\n' )`" is **verboten**, since it could theoretically run forever! Instead, you'll need to get into the habit of writing loops like that as "`while( ( (c=getc()) != '\n' ) && ( x++ < 1000 ) )`" (assuming `x` has already been declared and initialized to 0), or something similar.) Notice now that both tasks are wrapped in an `if` statement that doesn't block; if either task has no work to do (i.e. if either expression evaluates to `false`), then the superloop simply skips over it and goes on to the next task. (The Arduino IDE also has an example of this, under Examples > Digital > BlinkWithoutDelay.) Additionally, there is no line of code or function call inside either task that could run indefinitely.

There are two "gotchas" to be careful of here, though, both related to the fact that `g_systick` and `PD_next_run` will eventually overflow and wrap back around to 0.

First, the equality comparison for the `PD` task (`if( PD_next_run - g_systick >= PD_period_ms )`) is very carefully constructed. It is written to ensure correct behavior even when `g_systick` and `PD_next_run` eventually overflow and wrap back around to 0. Consider, for example, the possibility that `PD_next_run` has a value like `UINT_MAX` and `g_systick` has a value like 0. In that case, `g_systick` has wrapped around and we *want* the `PD` task to run. A simple comparison like `if( g_systick > PD_next_run )` would result in erroneous behavior, though, since 0 is not greater than `UINT_MAX`. Instead, the comparison used above essentially *incorporates* wraparound by asking, "What value would need to be added to `g_systick` to exceed `PD_next_run` and is this value greater than `PD_period_ms`?"

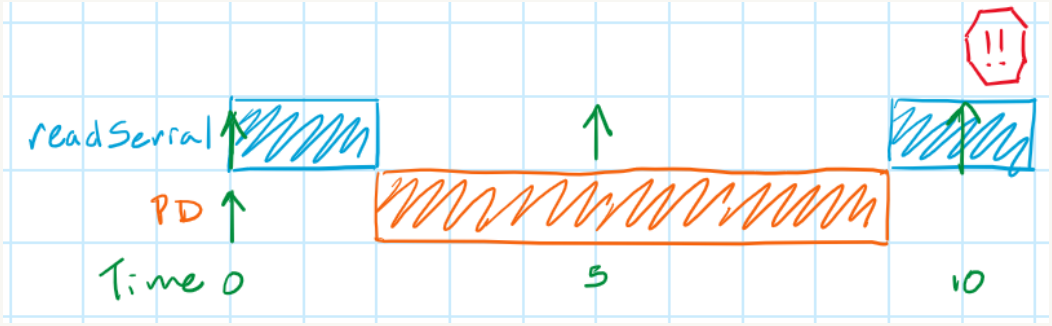
Second, an `unsigned int` on this hypothetical machine is assumed to be large enough that it's unlikely to wrap around fully every time through the `while(1)` loop. (Put another way, the execution times of the two tasks are assumed to be short relative to the timer period). This is important to ensure that the timer value looks monotonic to each task (i.e. the timer value increases gradually every time through the loop). Consider the case where an `unsigned int` is only 8-bits and where the `readSerial` task takes 257 "ticks" of the system clock in between the `PD` task running. In that scenario, to the `PD` task, it would *seem* as if only 1 tick of the clock has elapsed (for an 8-bit number `val`, `val + 257 == val + 1`), which is clearly wrong. This is probably not a concern on most systems, but since our main motivation in using a superloop is to avoid erroneous behavior, I felt it was worth mentioning. To eliminate this error, you can either (1) keep the timer period longer than the total execution time of your tasks or (2) use a

larger sized integer. If the size of your integer exceeds the width of your processor, though, you'll need to make sure that any reading of the `g_systick` variable in your main code is atomic. Otherwise you run the risk of a "torn read", which could happen if the timer ISR were to run (and `g_systick` updated) in the middle of the main code reading each of the bytes in `g_systick`. (The easiest way to ensure that a multi-byte read is atomic is to turn off interrupts before reading the `g_systick` variable and re-enabling them after.)

Lastly, let's put some number to these schedulers! Let's assume that the `readSerial` task takes, at worst, 2 us to complete (which includes any time spent in `processCommand()`) and that it needs to handle at least 200,000 characters a second. This means that, once a character is ready to be read, there may be another character coming in only 5 us (1/200000) so the `readSerial` task *must* complete before then, otherwise an incoming character might get dropped. Next, let's assume that the `PD` task takes, at worst, 7 us to complete and that we need this controller to run at 67 kHz, which means it needs to run every 15 usec. Additionally, we assumed when we built our PD model that the controller would get updated every 15 usec *exactly*. If there is too much **jitter** (i.e. variance) in the actual updates to the controller then it might not even work. Therefore we'll say that the `PD` task (in addition to it needing to run once every 15 usec) will need to finish within 13 usec of the start of the period. We can represent this information in a table, like that below.

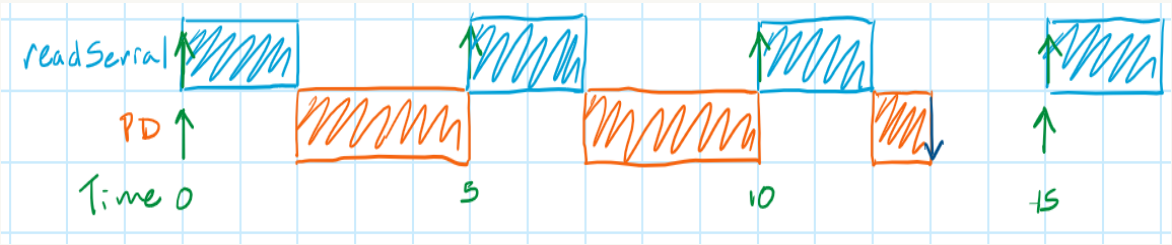
Task	Execution time (us)	Period (us)	Deadline (us)
readSerial	2	5	5
PD	7	15	13

We can show system load using a modified sequence diagram, like the one below. This type of diagram shows how a scheduler would execute the tasks above and, very importantly, how that relates to each tasks' deadline. Colored blocks indicate that a certain task is being executed on the processor at that moment, one color and row for each individual task (we'll assume our system is running on a processor with only one core for the entirety of this series). Upward-facing green arrows indicate that a task has been released (i.e. it's ready to run). Bad things happen (e.g. the PD controller doesn't work or we miss a serial character) if the task is not completed before its next deadline, which is indicated with either a downward-facing blue arrow or the next upward-facing green arrow. The diagram below shows what would happen in this system if we used a superloop.



Rats! The superloop scheduler misses a deadline at 10 usec. At 2 usec the `PD` task started running and it hogs the processor until 9 usec, when it finally completes. The `readSerial` task starts running but doesn't have enough time to complete before another character arrives and is lost. Sad day!

Let's see how the RTOS would have fared.



Success! Instead of being blocked by the `PD` task, the `readSerial` task is allowed to run at 5 usec by preempting the `PD` task (owing to its higher task priority of 0, as indicated in the line of code that created the `readSerial` task: "`taskCreate(readSerial, 0); // Priority 0 (highest)`"; the `PD` task gets preempted again at 10 usec). The `PD` task doesn't actually get to complete until 13 usec (as opposed to 9 usec when the superloop was in charge), barely within its deadline, but this system actually works! Let's now talk about *why* it seems to work.

Definition Time

All embedded systems perform **tasks**, which are **released** or "triggered" to run by events such as expiring timers or user inputs once per **period** (we'll discuss later how to handle tasks that aren't exactly periodic). Every task also has a **deadline**, which is the amount of time after being released in which, if the task has not completed, "bad things happen". What is meant by "bad things" is system-dependent and can vary from "the GUI lags" to "the conveyor belt doesn't stop and the $10,000 machine wrecks itself" (or worse!). The former is what we will call a **soft deadline**, since it's *undesirable* but certainly not the end of the world if the bad thing happens. The latter is what we will call a **hard deadline**, since we don't *ever* want the bad thing to happen. (If the undesirability of the bad thing happening is somewhere in the middle, we'll call it a **firm deadline**.) In the case of the superloop above, the `readSerial` task missed it's deadline after being released by the arrival of a character (which was assumed to have arrived at 5 usec), causing that character to be lost (a bad thing), whereas the RTOS didn't, making it the better design choice (in this very simplified example).

What causes a task to miss it's deadline? It all comes down to the other tasks that are demanding the processor's attention. In the moment when a certain task is released, the processor may very well be executing another task. What happens next depends on (1) the type of scheduler being employed and (2) the relative priorities of the two tasks (and any other tasks which are ready and waiting to run). Schedulers can be divided (among many other ways) into one of two types: cooperative and preemptive. A **cooperative** scheduler relies on each task yielding control back to the processor before it can decide which task should run next; it cannot wrest control back

from any task. A **preemptive** scheduler *can* force a task to be put on "pause" in order to run another higher priority task; in that case the lower-priority task is said to be **preempted**. The superloop scheduler is a cooperative scheduler while the RTOS scheduler is (nearly always) a preemptive scheduler. If a cooperative scheduler is being used then the newly released task will first have to wait for any currently running task to complete, even if that task is of a lower priority. In either case, the scheduler will then run any other ready-to-run tasks in priority order: higher priority tasks are run before lower priority tasks. So the newly released task will also need to wait for any higher priority tasks to complete before it can run. (For our current superloop scheduler there isn't really a concept of "priority" yet, but the end result is the same: a task which is ready to run will have to wait for some number of other tasks to start and finish before its allowed to execute.) If a task is waiting for another task to complete before it is allowed to run, that task is said to be **blocked** and if a task is blocked for too long then it will miss its deadline.

The longest possible length of time that a certain task could demand the processor's attention (blocking all other lower priority tasks) is called it's **worst-case execution time (WCET)**. The WCET of a task comes about when a certain set of inputs causes the task to

- fall through its longest conditional paths,
- complete the maximum number of `for` or `while` loop iterations,
- call library functions that take the longest time to finish executing (printf, for example, can take more or less time to execute depending on the exact numerical value being converted into characters),
- and also, for more complex processors, trigger the maximum feasible number of cache misses, branch mispredictions, etc.

A tasks' **worst-case response time (WCRT)** is the longest amount of time that a processor might spend on other tasks (i.e. the longest time that a task could be blocked) *added to* the tasks own execution time (a task must finish executing before the deadline expires in order to avoid "bad things happening"). For a task to *never* miss it's deadline it must have a WCRT that is less than it's deadline. If this is the case for all tasks in a system, then the system is said to be **schedulable**.

Our set of tasks is not schedulable using the superloop scheduler because the WCRT of the `readSerial` task exceeds its deadline, as can be seen in the diagram. This same set of tasks *appears* to be schedulable using the RTOS by the same logic. But how do we *know* if a set of tasks is schedulable or not using a given scheduler? Time for some math! And before that, let's establish a few critical assumptions about our system to avoid making the math super gross.

Critical Assumptions

1. **All tasks are periodic**
   This means that we'll assume all tasks become ready to run at the start of their periods (like little metronomes) with no jitter, and all periods are known ahead of time and never

change. This is, actually, how the `PD` task above operated (the same as any other task released by a timer), but it's a weirder assumption for something like the `readSerial` task, which is event-driven (a.k.a. "sporadic"). For event-driven tasks, we'll just have to pretend that the task *is* periodic, with a "pseudo-period" equal to the fastest "inter-arrival" time of the events (the shortest time between any two subsequent events). This may mean that we over-engineer our system a little, but that's just the price we pay in order to ensure that our system still works, even under the heaviest of loads. (It's possible to analyze a system in which tasks are released in a "bursty" fashion, but doing so is quite outside the scope of this series.)

2. **All tasks' deadlines are less than or equal to their periods**
   This means that each task *must* finish executing *before* its next release time, like in our example system above. There can only ever be one pending invokation of a task, not multiple. Analyzing a system in which tasks are allowed to have deadlines that are longer than their periods (i.e. tasks that are enqueued somehow, or released multiple times before they are processed), is challenging enough that we'll save it for a future article.
3. **All tasks are completely independent**
   This means that no task ever blocks another from executing (as might happen if one task were using a shared hardware resource or the mutex to a shared variable). This *does* happen in real life, of course, but not in any deterministic sort of way that we could reasonably include in our analysis. So the best that we can do is assume that there isn't any blocking time and then try to design our systems to actually acheive that as best we can.
4. **It takes zero time for the scheduler to switch between tasks**
   This is not strictly true, but it's not far off, especially for well-written schedulers.
5. **Tasks are being executed on a single-core processor**
   We will not address any form of multi-processing in this series.

In reality, of course, none of these are true all the time which, if you're anything like me, is infuriating. What this means to me, though, is that what we're really doing here is creating a system for which the chances of a task missing its deadline are supremely low. By being overly pessimistic in our estimation of each tasks' WCET and WCRT, we're giving ourselves some headroom to account for the fact that our assumptions (above) aren't exactly true all the time.

The Schedulability of a Superloop

So a set of tasks is schedulable using a given scheduler if the WCRT of each task is less than it's respective deadline. We'll use an inductive technique to arrive at an equation for the WCRT of a task: by assuming that a task has become ready to run *immediately after* the scheduler has determined that it was *not* ready to run (and, subsequently, decided to run the next task).

For our example, this worst-case scenario would occur for the `readSerial` task if, immediately *after* the expression "`if( charAvailable() )`" were evaluated (and the `readSerial` task skipped), the `readSerial` task became ready to run. What series of events could then transpire that would cause the `readSerial` task to have to wait the longest before it could run? That would simply be if

every other task was also ready to run *and* took their *longest possible* execution paths (their WCETs) when they did. So the WCRT of any task in this superloop scheduler is equal to the sum of the WCETs for all other tasks in the system plus its own, which is the same as the sum of the WCETs for all of the tasks in the system. We can express this in equation (1) for a set of *N* tasks, where each task has a unique number from *0* to *N-1*. (In the future, a task's number will also correspond to its priority, with Task 0 having the highest priority and Task N-1 having the lowest priority.) Thus, the notation *i!=j* means "for all tasks that are not task j".

$$WCRT_j = (\sum_{i!=j}^{N-1} WCET_i) + WCET_j = \sum_{i=0}^{N-1} WCET_i \quad (1)$$

This is true of every task in the system. So if the sum of the WCETs for *all* tasks in this system is *less than the shortest* deadline, then *every* task will be able to meet its deadline; that set of tasks is said to be schedulable using a superloop. Notice that this requirement (that the sum of the execution times is less than the shortest deadline) is easy to meet for a set of tasks that (1) don't have long WCETs or that (2) run infrequently (i.e. have large periods), but harder if either of those aren't true.

How does this look for our example? Since the WCETs of the `readSerial` and `PD` tasks are 2 and 7 usec, the WCRT of either task is 9 usec. Since this is longer than the deadline for the `readSerial` task, we can determine that that set of tasks is **not schedulable** using a superloop. (In what scenario would the `readSerial` task have to wait 9 usec to complete? Imagine that neither task is ready to run and the scheduler is spinning in a loop, checking each `if` expression and seeing them as `false`. This, effectively, randomizes the point at which each task is evaluated to see if it's ready to run. Imagine in that scenario that the clock ticks over and the `PD` task is ready to run. A fraction of a clock tick later, a character arrives to be processed by `readSerial`. In that situation, the `PD` task would run for 7 usec, the `readSerial` task would run for 2 usec, and if a new character arrived before these were both complete then that character might be lost. Hence, we have a WCRT for either task of 9 usec.)

Measuring WCET

Accurately knowing the WCET for a task is clearly a critical prerequisite to ensuring that the system you're designing is schedulable. There are essentially two ways to do that:

1. Look at the assembly code that's generated for a task and count how many clock cycles the longest execution path would take.
2. Run the task in question while attempting to exercise it such that it takes the longest execution path and then measure how long it takes.

The first method will likely yield the most accurate value but its tedious, and possibly even intractable if the task includes library functions for which you don't have the source code. The second method is much easier but always holds the chance that the *absolute* longest execution path isn't actually measured (can you *guarantee* that your code has executed the longest code path? That your *library code* has executed the longest code path? That you've triggered the

maximum feasible number of cache misses or branch mispredictions?). Using this method usually involves multiplying the longest time the task in question *actually* takes by some value slightly more than 1, to incorporate a bit of a safety margin.

There are two primary ways to measure the execution time of a piece of code:

1. Using a logic analyzer or oscilloscope
2. Using a serial converter or debug adapter

A third option also exists for RTOS-based systems: Using a program like Tracealyzer from Percepio to determine the execution time of each task.

Using a logic analyzer or oscilloscope

In this method, you'll toggle a GPIO at the start and end of the piece of code and then observe the resulting pulse using a logic analyzer or oscilloscope. (The period of the pulse is also a measure of how often that task gets run, which is useful information.) It's most helpful if you can configure your logic analyzer or oscilloscope to record multiple traces or otherwise help you see the longest pulse so that you're not just watching the numbers for the pulse length flash by and hoping that you'll glimpse the highest value.

Using a serial converter or debug adapter

"Why use an external timing device," you say, "when our MCU is rife with timers?" An excellent question! In this method, you'll configure one of the MCU's timers to increment every nanosecond or microsecond or millisecond (whichever is most appropriate for your tasks) and log the value of the timer at the start and end of the piece of code. (Some devices may have a running counter of clock cycles, which would also work, you'll just need to remember to convert your timing values later on from something like "42 clock cycles" to "875 nsec".) At the end of the piece of code, once the execution time is known, you can keep track of just the highest value or even add each value to a histogram to get an idea of the distribution of execution times for the piece of code in question.

There are many ways to get the data off of your MCU:

- Stream each value out of the serial port (or just send out the largest value once a second).
- Periodically halt the processor using a debug adapter and inspect the variable holding the largest value.
- Use a program like uC/Probe or STM32CubeMonitor to continuously display the largest value.

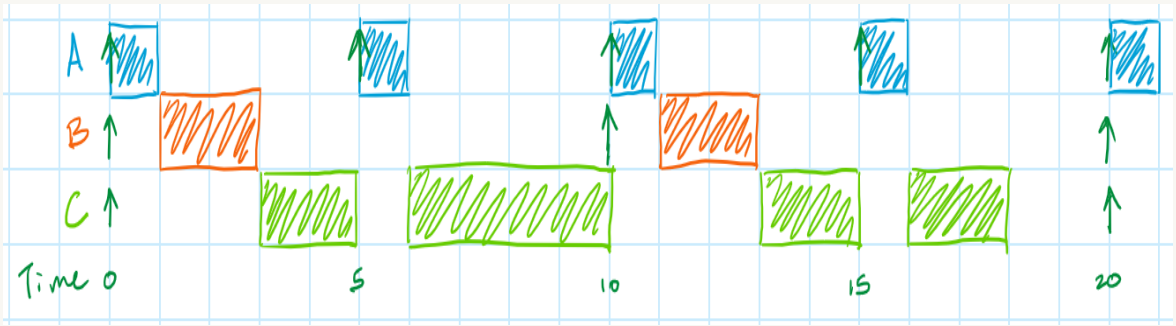
Don't forget to account for rollover of the timer when you're calculating the execution times, though! Writing this code is left as an exercise for the reader. ;) (Man, I've *always* wanted to say that! JK, I can provide some starter code if you ask for it in the comments.)

Schedulability of an RTOS

Now let's move on to the RTOS scheduler. Again let's ask the question: "Once a task has become ready to run, what's the worst possible series of events could then transpire that would cause that task to have to wait the longest before it could run?" Since this is a preemptive scheduler we needn't worry about the WCETs of any lower priority tasks; the scheduler should preempt any of them in order to let the task in question run. *However*, we do still need to concern ourselves with any higher-priority tasks that are running or are ready to run; our task in question *will* need to wait for them to complete before it can run. Further, for this to be a "worst case" analysis, we need to assume (as we did above) that all of those tasks take the execution paths that result in their WCETs. So the WCRT of any task in this RTOS scheduler is equal to the sum of the WCETs for all other *higher-priority* tasks in the system plus their own, as shown in equation (2). Here, the the summation from 0 to *j - 1* means "for all tasks that have a higher priority than task j" (remember that a lower task number, per our nomenclature, indicates a higher task priority).
$$WCRT_j = \left(\sum_{i=0}^{j-1} WCET_i\right) + WCET_j \quad (2)$$

But wait! We can't just add the WCETs of each higher-priority task *once*; that would only be true if the system were designed such that each higher-priority task could only run once within a given period. In actuality, the higher-priority tasks may be released multiple times before the task in question gets to finish, which means that our summation in the equation above is *recursive*: if each of the higher-priority tasks are released when the task in question is ready to run and, in the process of running, some of those higher-priority tasks are released *again*, then we need to add that second execution into our value for the WCRT of the task in question (and so on if, during *that* execution, *even more* higher-priority tasks are released). Consider, for example, the task set below:

Task	Priority	WCET (us)	Period (us)	Deadline (us)
A	0	1	5	5
B	1	2	10	10
C	2	10	20	20



At 0 sec, all tasks are ready to run. Task A has the highest priority so it runs first, followed by Task B, and then Task C starts. This means that when calculating the WCRT of Task C, we will

need to include one iteration each for Tasks A and B (so far). Before Task C finishes, though, it gets *preempted* by Task A at 5 usec; a *second* iteration of Task A. Task C gets interrupted again at 10 usec (by Tasks A and B) and at 15 usec (by Task A, for the last time). So Task C's final WCRT is **18 usec**, owing to all of the times it was blocked or preempted by the higher-priority tasks A and B.

A more correct version of equation (2) is shown below, which says that the WCRT of a task using a preemptive RTOS is equal to the WCET of the task plus the total number of times that any higher-priority task might be released before the task in question finishes execution (those vertical bars are depicting the **ceiling function**, they're not brackets).

WCRTj=∑i=0j−1(⌈WCRTjPi⌉·WCETi)+WCETj(3)

Notice that the term we're trying to solve for, $WCRT_j$, is on *both sides* of the equation. This is what I meant by this equation being recursive: every time a higher-priority task runs, extending a task's WCRT, it gives *other* higher-priority tasks additional opportunities to run. Solving for the WCRT of Task C requires guessing a value for the left-hand side of equation (3) (I'll start with the *very* optimistic assumption that $WCRT_C = WCET_C$) and checking to see if it matches the right-hand side of equation (3). If it doesn't, you'll use the value from the right-hand side of equation (3) as the new WCRT and repeat. Here's how that would if we were to work that out numerically for Task C from the task set above: 10=?⌈105⌉·1+⌈1010⌉·2+10=2·1+1·2+10=14(4) The left-side (10) doesn't equal the right side (14), so let's try again, this time using 14 for WCRT $_C$. 14=?⌈145⌉·1+⌈1410⌉·2+10=3·1+2·2+10=17(5) Still no convergence, so let's use the new value of 17 usec. 17=?⌈175⌉·1+⌈1710⌉·2+10=4·1+2·2+10=18(6) Keep going! 18=?⌈185⌉·1+⌈1810⌉·2+10=4·1+2·2+10=18(7) Success! The WCRT of Task C is 18 usec, which agrees with our simulation above.

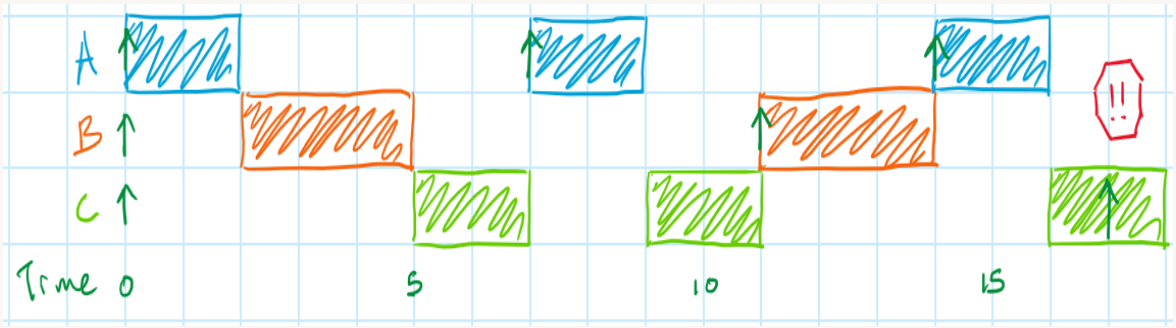
So the process of computing the WCRT of a task using a preemptive RTOS looks like this:

1. Assume a value for the WCRT of a task (such as it's WCET).
2. Figure out how many times each of the higher-priority tasks in the system would be released during the time it would take the task in question to finish executing (rounding up) and add those to the WCET of the task in question.
3. Use this value as the new WCRT for the task in question and go back to step 2 until your guessed value for the WCRT matches the right-hand side of equation (3).

This is true of all tasks in the system. As above, if the WCRTs using the above equation are shorter than the deadlines for each of the tasks, then the system is schedulable. In this case, though, as opposed to the superloop, the WCRTs are clearly much shorter for the higher-priority tasks. This makes certain task sets schedulable using the RTOS that wouldn't be schedulable using the superloop. (Interestingly to me, though, the RTOS doesn't improve the WCRT of each task universally; the WCRT of the lowest-priority task is the same as for, or possibly even *worse* than, the superloop, and the WCRTs of the next higher-priority tasks might only be marginally

better than for the same tasks in a superloop.) Additionally, there are some task sets that are simply not schedulable, even with an RTOS. For example, consider the task set below.

Task	Priority	WCET (us)	Period (us)	Deadline (us)
A	0	2	7	7
B	1	3	11	11
C	2	6	17	17



Even though overall CPU utilization is only 91.1% (utilization is defined below), Task C has a response time of 18 usec! As can be seen in the figure, Task C misses its deadline at 17 usec because the system was so busy with Tasks A and B that Task C wasn't able to finish. From this I think it's important to understand that an RTOS scheduler is not a panacea; in fact, there are *some* task sets that *aren't* schedulable with an RTOS but *are schedulable* with a superloop! We'll discuss *those* situations in a future article.

Assigning task priorities

Which task in an RTOS-based system has the highest (or lowest) priority is clearly a very important decision. For RTOS-based systems that adhere to our Critical Assumptions above, it turns out that assigning priority based on each tasks' deadline is optimal. This method is called **deadline monotonic shceduling** (DMS). This approach assigns the tasks with the *shortest* deadlines the *highest* priorities, in order. If you look back at the two sets of tasks that were analyzed in the Schedulability of an RTOS section, you'll notice that the priority of each task was assigned using this exact method. This isn't the *only* way to assign priorities, but the fact that this method is "optimal" means that if *any* other set of priorities allows a set of tasks to be schedulable, then DMS is guaranteed to also be schedulable.

The Utilization Test and Harmonic Rate Monotonic Scheduling

Much ink has been spilled about the merits and qualities of RTOS schedulers, and one neat discovery to come from that research is that there is a simple test to see if a set of tasks is schedulable using an RTOS that *doesn't* involve adding up (recursively) a bunch of WCETs and comparing each individual WCRT for each task against its deadline. That test is called a

"utilization test" because it simply relies on how much of the processor is utilized by each of the tasks.

The first step in applying this test is to calculate how much of the processor is utilized by a given task set. To do that, simply divide each task's WCET by its period and then add all of the percentages together.

$$Utilization = \sum_{i=0}^{N-1} \frac{WCET_i}{Period_i} \quad (8)$$

A neat application of this is that a set of N tasks is guaranteed to be schedulable with a preemptive RTOS if the processor utilization is less than the value given by equation (9), below.

$$Utilization \leq N(2^{1/N} - 1) \quad (9)$$

This doesn't exactly work, though, if a task's deadline is *less than* its period. That's a more restrictive scenario than if the deadlines were all equal to each task's period. In that case, it's possible to create a "pseudo-period" for the task which is equal to its (smaller) deadline. This overestimates how often the task will run but does allow us to use equation 9.

This condition is *sufficient* but not *necessary* for a task set to be schedulable. What that means is that if processor utilization is *above* the value in equation (9) (but less than or equal to 100%, of course), it doesn't *automatically* mean that the task set *isn't* schedulable. In that case, what you need to do is calculate the individual WCRTs of each task (recursively, as described above) to determine if the set of tasks is schedulable.

Additionally, processor utilization can be as high as *100%* if the task periods are all *harmonic*, or integer multiples of each other. Additionally, this requires that all tasks' deadlines be **equal to** their periods, which is a stricter requirement than we set above in the section Critical Assumptions. A task set with periods of 1, 5, 10, and 20 usec is harmonic (because 1 divides evenly into 5, 10, and 20; 5 divides evenly into 10 and 20; and 10 divides evenly into 20) but a task set with periods of 1, 5, 10, and *15* are *not* harmonic (since 10 doesn't divide evenly into 15). For some systems, this means that you can actually make a set of tasks schedulable (when it wouldn't be otherwise) by running some tasks *faster* than is necessary; processor utilization could be as high as 100% if the 15 ms task above were to be run at 10 ms.

Summary

Regardless of the embedded application you're building, your system is composed of tasks that are released by either a timer or some kind of external input and which need to complete their work before their deadlines to avoid "bad things" from happening. The scheduler decides which task runs when and is a critical part of designing a system that will meet all of its deadlines. In this article, I showed you how to do just that by:

1. Determining each task's worst-case execution time (WCET), period, and deadline. The WCETs can be calculated by analyzing source code or measured by either toggling a GPIO

(and observing the pulse using a logic analyzer) or by storing the start and end times of a task (using a hardware timer) and piping the data out using a serial adapter or debug adapter. The period and deadlines for each task are set by you, the designer, or by other system constraints.
2. Determining if the task set is schedulable using a superloop by adding up the total WCETs (equation 1) and comparing this to the shortest deadline of any task **OR**
3. Determining if the task set is schedulable using an RTOS by calculating the utilization of the task set (equation 8) and comparing this value to the upper limit set by equation (9) or by comparing this value to 1 (if your task periods are all harmonic and every task's deadline equals its period). If the utilization is **above** the upper limit set by equation (9), than you must calculate the WCRTs of each task using equation (3) and comparing each task's WCRT to its deadline.
4. If you plan on using an RTOS, then, lastly, you will assign task priorities via deadline monotonic scheduling (DMS).

Additionally, and perhaps of most importance to me, you've seen the problems that preemptive schedulers can create for system designers (race conditions... ew...).

I hope you enjoyed this introduction to real-time scheduling, learned a thing or two today, and are excited to see how the humble superloop could ever dethrone the king of real-time scheduling, the RTOS! Stay tuned, my friends!

References

- Better Embedded System Software by Philip Koopman
- Embedded Systems Fundamentals with ARM Cortex-M based Microcontrollers, Chapter 3, by Alexander Dean
- A lot of my notes come from taking Professor Alex Dean's classes at NC State: ECE560 (Embedded Systems Architectures) and ECE561 (Embedded Systems Design/"Optimization"). His course notes reference, among others, the following articles and books:
    - George, L., Rivierre, N., & Spuri, M. (1996). *Technical Report RR-2966: Preemptive and Non-preemptive Real-time Uniprocessor Scheduling.* INRIA.
    - Sha, L., Abdelzhaer, T., Arzen, K.-E., Cervin, A., Baker, T., Burns, A., et al. (2004, November-December). Real Time Scheduling Theory: A Historical Perspective. *Real Time Systems*, 28(2-3), 101-155.
    - Davis, R. I., Thekkilakattil, A., Gettings, O., Dobrin, R., Punnekkat, S., & Chen, J.-J. (2017). Exact speedup factors and sub-optimality for non-preemptive scheduling. Real-Time Systems, 54(1), 208–246. https://doi.org/10.1007/s11241-017-9294-3
    - Real-time Systems and Programming Languages by Alan Burns and Andy Wellings
    - A Practitioner's Guide Handbook for Real-time Analysis by Klein, Ralya, Pollak, Obenza, and Harbour.